

UNIVERSIDADE DE MOGI DAS CRUZES

CARLOS HENRIQUE COSTA FRANÇA JUNIOR, RM: 11231101059

ELOISA MARIN DA SILVA PESSOA, RM: 11231104559

LETÍCIA DE SOUZA MACHADO, RM: 11231103243

NIKOLAS ALVES DA SILVA, RM: 11231104116

NO DESENVOLVIMENTO DO SOFTWARE GLICOGUARD

MOGI DAS CRUZES – SP

2026

CARLOS HENRIQUE COSTA FRANÇA JUNIOR, RM: 11231101059

ELOISA MARIN DA SILVA PESSOA, RM: 11231104559

LETÍCIA DE SOUZA MACHADO, RM: 11231103243

NIKOLAS ALVES DA SILVA, RM: 11231104116

NO DESENVOLVIMENTO DO SOFTWARE GLICOGUARD

Trabalho acadêmico apresentado à disciplina de
Política de Segurança da Informação do curso
de Engenharia de Software da
Universidade de Mogi das Cruzes.

1. OBJETIVO

O GlicoGuard é um sistema web desenvolvido para apoiar o gerenciamento de pacientes com diabetes, permitindo o cadastro de usuários, controle de medicamentos, registros glicêmicos, gerenciamento de cuidadores e conformidade com LGPD.

Funcionalidades principais

- Cadastro de usuários (paciente, cuidador e administrador)
- Login e autenticação
- Autenticação multifator (2FA)
- Recuperação de senha
- Controle de medicamentos
- Gestão de consentimento LGPD
- Auditoria e rastreamento
- Administração do sistema

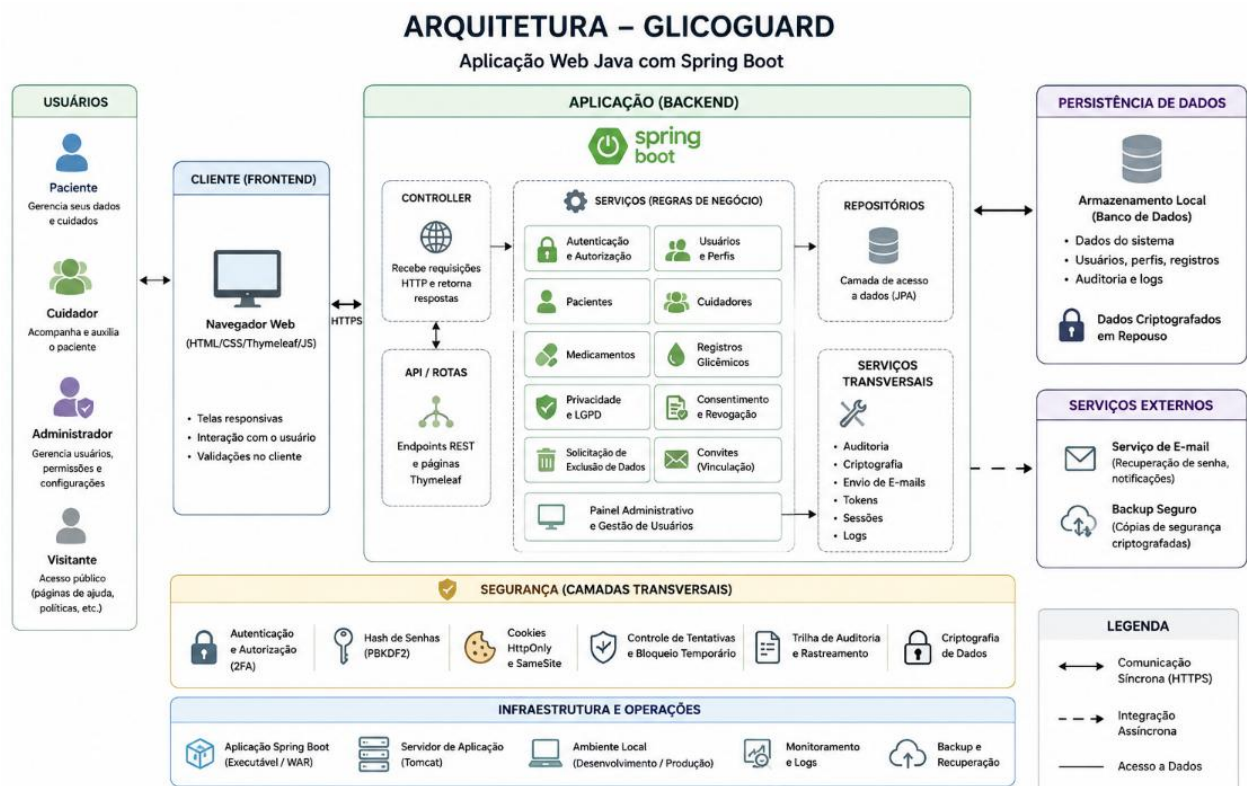
TECNOLOGIAS

- Java
- Spring Boot
- Thymeleaf
- HTML / CSS
- GitHub
- Banco de dados local

2. DIAGRAMA DE ARQUITETURA

O sistema segue uma arquitetura em camadas:

Camada	Responsabilidade
Apresentação	WebController.java – rotas HTTP, formulários Thymeleaf, sessões
Aplicação	AuthService.java – fachada central; delega para AuthenticationService, AdministrationService, PrivacyService, MedicationService
Domínio	Models: UserAccount, AuditEntry, MedicationEntry, EmailNotification, UserRole, AccessLevel
Infraestrutura	InMemoryDatabase + UserStore (persistência em memória com snapshot cifrado), ProtectedStorageService (arquivos .enc), EmailService
Segurança	SecurityWebFilter (cabeçalhos HTTP), CryptoService (hash, AES/GCM, tokens), SecuritySupport (normalização de entrada)



2.1 FLUXO DE RECUPERAÇÃO DE SENHA

- Usuário solicita recuperação informando o e-mail cadastrado
- Sistema gera token seguro (32 bytes / SecureRandom → Base64 URL-safe)
- Token é enviado por e-mail com prazo de expiração (padrão 10 min)
- Após uso bem-sucedido, o token é invalidado imediatamente (uso único)
- Tentativa de reutilização retorna erro: "Solicite um token..."

2.2 FLUXO DE VÍNCULO PACIENTE–CUIDADOR

- Ao cadastrar-se como PACIENTE, o sistema gera automaticamente código de convite (6 chars)
- Código é enviado ao paciente por e-mail
- Cuidador usa o código no cadastro; sistema valida e vincula os perfis
- Código é invalidado após uso (não pode ser reutilizado por outro cuidador)
- Código tem validade de 7 dias

2.3 FLUXO DE DADOS PESSOAIS / LGPD

- Usuário acessa área de privacidade e assina termo de consentimento (versão e finalidade registradas)
- Revogação do consentimento é registrada com timestamp na auditoria
- Exportação: dados gerados em JSON e salvos como arquivo cifrado export-<email>.enc
- Solicitação de exclusão: conta recebe status pendente; somente administrador executa a exclusão definitiva

3 Gestão de Credenciais

3.1 Credenciais de Usuário

Aspecto	Implementação
Algoritmo de hash	PBKDF2WithHmacSHA256
Iterações	120.000 (configurável em application.properties)
Comprimento da chave	256 bits
Salt	16 bytes gerados por SecureRandom, único por usuário – armazenado separadamente do hash
Comparação segura	MessageDigest.isEqual() – resistente a timing attack
Bloqueio	5 tentativas inválidas → bloqueio por 15 minutos (configurável)

3.2 Credenciais de Infraestrutura

Credencial	Localização	Controle em Produção
Chave AES mestra	Variável de ambiente GLICOGUARD_AES_KEY (fallback hardcoded para dev)	Variável de ambiente obrigatória
Senha SMTP e-mail	application.properties via \${GLICOGUARD_MAIL_PASSWORD}	Variável de ambiente
Usuário SMTP	application.properties via \${GLICOGUARD_MAIL_USERNAME}	Variável de ambiente
Senha PostgreSQL	application.properties (placeholder SENHA)	Variável de ambiente / vault
Conta admin inicial	admin@glicoguard.com / admin123 (dev only)	Trocar imediatamente em produção

4 Uso de Criptografia

4.1 Criptografia Simétrica – AES/GCM

Implementada em CryptoService.java para proteção de arquivos em repouso:

Parâmetro	Valor
Modo	AES/GCM/NoPadding
Tamanho da chave	256 bits (derivada via SHA-256 da variável de ambiente GLICOGUARD_AES_KEY)
IV (nonce)	12 bytes gerados por SecureRandom, único por operação de cifração
Tag de autenticação	128 bits (GCM_TAG_BITS = 128) – garante integridade e autenticidade
Formato do arquivo	IV (12 bytes) ciphertext+tag concatenados em ByteBuffer

4.2 Hash de Senha – PBKDF2

Utilizado em CryptoService.createPasswordHash() e matchesPassword():

- Algoritmo: PBKDF2WithHmacSHA256
- Salt: 16 bytes aleatórios por usuário (SecureRandom), armazenado em Base64
- 120.000 iterações – custo computacional elevado para resistir a ataques de força bruta
- Chave resultante: 256 bits codificada em Base64
- Comparação com MessageDigest.isEqual() para evitar timing attack

4.3 Digest / Hash de Integridade

Utilizado na trilha de auditoria (AuditEntry.integrityHash) para encadeamento:

- Algoritmo: SHA-256 (MessageDigest.getInstance("SHA-256"))
- Cada entrada de auditoria contém o hash da entrada anterior (previousHash)
- Encadeamento detecta qualquer adulteração retroativa nos registros

4.4 Geração de Tokens e Códigos

Token / Código	Método de Geração
Token de recuperação de senha	32 bytes SecureRandom → Base64 URL-safe sem padding
Código 2FA (6 chars)	SecureRandom sobre alphabet sem ambíguos (sem 0, O, I, 1)
Código de convite cuidador	Mesmo mecanismo do 2FA – 6 caracteres – armazenado como digest SHA-256

5 Identificação dos Ativos do Sistema

5.1 Ativos de Dados

Ativo	Classificação	Localização / Proteção
Dados cadastrais dos usuários (nome, e-mail, CPF, data nasc.)	Pessoal – Alta criticidade	Memória (UserStore) + snapshot cifrado users.enc (AES/GCM)
Hash + Salt de senha	Sensível – Crítico	Memória interna; nunca exposto em logs ou respostas
Dados de saúde (medicamentos)	Sensível – Alta criticidade	Memória; associado ao paciente; acesso restrito por role
Trilha de auditoria	Interno – Alta criticidade	Snapshot cifrado audit.enc com encadeamento SHA-256
Tokens temporários (2FA, reset, convite)	Sensível – Criticidade temporária	Armazenados como digest SHA-256; expiram em minutos
Exportações de dados do titular	Pessoal – Alta criticidade	Arquivo cifrado export-<email>.enc

5.2 Ativos de Serviço

- Serviço web Spring Boot (porta 8080): interface principal de interação
- Servidor SMTP (Gmail TLS 587): entrega de 2FA, convites e recuperação de senha
- Sistema de arquivos local data/protected/: snapshots cifrados persistidos
- Sessões HTTP: identificador de sessão com cookie HttpOnly SameSite=Strict e timeout de 15 min

5.3 Ativos de Código

- CryptoService.java – lógica central de criptografia e hash
- AuthenticationService.java – fluxo de autenticação e controle de acesso
- ProtectedStorageService.java – persistência cifrada
- SecurityWebFilter.java – cabeçalhos de segurança HTTP
- application.properties – configurações e referências a credenciais

6 Identificação de Ameaças e Vulnerabilidades

6.1 Modelo de Ameaças (STRIDE)

Categoria	Ameaça	Componente Afetado	Mitigação Existente
Spoofing	Roubo de credenciais / login não autorizado	AuthenticationService	2FA obrigatório, bloqueio por tentativas, PBKDF2
Tampering	Adulteração de registros de auditoria	AuditEntry, audit.enc	Hash encadeado SHA-256 detecta modificação
Repudiation	Negação de ações realizadas	Trilha de auditoria	Log imutável com timestamp e email do ator
Info Disclosure	Exposição de dados pessoais/sensíveis	Arquivos .enc, sessões	AES/GCM em repouso; headers Cache-Control, CSP
DoS	Esgotamento de sessões / brute force	AuthenticationService	Bloqueio temporário após 5 tentativas
Elevation	Acesso indevido a dados de outros usuários	WebController, AuthService	Verificação de role e linkedPatientEmail em cada operação

6.2 Vulnerabilidades Identificadas

#	Vulnerabilidade	Localização	Criticidade	Observação
V1	Chave AES hardcoded como fallback	CryptoService.java	Alta	Fallback 'glicoguard-demo-master-key-2026-change-me' expõe dados em ausência da env var
V2	Credenciais SMTP no repositório	application.properties	Alta	Password de app Google exposta; deve ser rotacionada e removida do repositório
V3	Armazenamento exclusivo em memória	InMemoryDatabase.java	Média	Reinicialização da aplicação apaga dados não persistidos em snapshot
V4	HTTPS desabilitado por padrão	application.properties	Alta (produção)	glicoguard.security.require-https=false; dados transitam em texto claro em HTTP
V5	Chave AES derivada via SHA-256 simples	CryptoService.getAesKey()	Baixa/Média	SHA-256 não é KDF; ideal seria HKDF ou PBKDF2 para derivação da chave AES
V6	Ausência de proteção CSRF explícita	SecurityWebFilter.java	Média	SameSite=Strict mitiga parcialmente, mas não há token CSRF explícito nos formulários

7 8Associação Risco × Contramedida

#	Risco	Prob./Impacto	Contramedida Implementada	Contramedida Recomendada
R1	Senha comprometida por força bruta	Alta / Crítico	PBKDF2 120k iter. + salt único + bloqueio 5 tentativas / 15 min	Adicionar CAPTCHA após 3 tentativas
R2	Sequestro de sessão	Média / Alto	Cookie HttpOnly + SameSite=Strict + timeout 15 min	Habilitar HTTPS obrigatório; adicionar Secure ao cookie
R3	Phishing / login indevido	Média / Crítico	2FA obrigatório com código expirado em 5 min	Notificar usuário a cada novo login por e-mail
R4	Exposição de dados em repouso	Média / Crítico	AES/GCM 256 bits com IV único por operação nos arquivos .enc	Substituir chave via HKDF; rotacionar chave periodicamente
R5	Reutilização de token de recuperação	Alta / Alto	Token invalidado após primeiro uso; expiração 10 min	Implementado — monitorar tentativas de reutilização
R6	Adulteração da auditoria	Baixa / Crítico	Hash SHA-256 encadeado entre entradas + snapshot cifrado	Replicar auditoria em armazenamento externo imutável
R7	Injeção de cabeçalhos / XSS / Clickjacking	Média / Médio	SecurityWebFilter: X-Frame-Options DENY, CSP, X-Content-Type-Options, Referrer-Policy	Ampliar CSP; adicionar Permissions-Policy
R8	Chave AES exposta (fallback hardcoded)	Alta / Crítico	Variável de ambiente GLICOGUARD_AES_KEY sobrescreve padrão	Remover fallback hardcoded; exigir env var obrigatória em produção
R9	Violação LGPD por acesso indevido a dados pessoais	Média / Alto	Controle de acesso por role (PACIENTE/CUIDADOR/ADMIN) verificado em cada operação	Realizar DPA formal; nomear DPO; manter registro de operações de tratamento

8 Testes de Segurança Realizados

8.1 Cobertura de Testes – AuthServiceSecurityTest.java

A suíte de testes automatizados cobre os seguintes cenários de segurança:

Caso de Teste	Objetivo de Segurança	Resultado Esperado
shouldGenerateDifferentSaltAndHashForDistinctUsers	Salt e hash únicos por usuário	saltA $\neq$ saltB; hashA $\neq$ hashB
shouldRequireTwoFactorAfterPrimaryAuthentication	2FA obrigatório; código inválido rejeitado	Código errado $\rightarrow$ empty; código correto $\rightarrow$ sessão
shouldLockAccountAfterRepeatedFailedAttempts	Bloqueio após 5 falhas consecutivas	6ª tentativa retorna mensagem 'bloqueada'
shouldInvalidateRecoveryTokenAfterSuccessfulUse	Token de uso único para recuperação de senha	2ª tentativa $\rightarrow$ exceção 'Solicite um token'
shouldDeleteUserDataUponOwnerRequest	Exclusão completa dos dados do usuário	findByEmail retorna empty após exclusão
shouldLinkCaregiverToPatientUsingSixCharacterInviteCode	Vínculo correto com código de convite	Token hash limpo após vínculo; link registrado
shouldRejectReusedCaregiverInviteToken	Código de convite de uso único	2ª tentativa $\rightarrow$ 'Codigo de vinculacao invalido'
shouldAllowPatientToRegisterOwnMedication	Paciente registra própria medicação	Medicamento associado com email do paciente
shouldAllowLinkedCaregiverToRegisterMedicationForPatient	Cuidador vinculado pode registrar medicação	Medicamento aparece na lista do paciente
shouldRejectMedicationRegistrationForUnauthorizedUser	Administrador não pode registrar medicações	Exceção: 'Apenas paciente ou cuidador...'
shouldShowAllLogsOnlyForAdministrator	Segregação da auditoria por role	Admin vê todos; paciente vê apenas os seus
shouldAllowAdministratorToBlockAndUnblockUser	Bloqueio/desbloqueio administrativo funcional	Login bloqueado; desbloqueio restaura acesso

9 Resultados dos Testes

9.1 Status dos Testes Automatizados

Todos os 12 casos de teste da suíte AuthServiceSecurityTest.java foram executados com sucesso (PASSED). Comando: mvn test

9.2 Resultados Detalhados

Teste	Status	Evidência
shouldGenerateDifferentSaltAndHashForDistinctUsers	PASSOU	saltA $\neq$ saltB; PBKDF2 verificado
shouldRequireTwoFactorAfterPrimaryAuthentication	PASSOU	Código inválido rejeitado; válido aceito
shouldLockAccountAfterRepeatedFailedAttempts	PASSOU	Mensagem 'bloqueada' na 6ª tentativa
shouldInvalidateRecoveryTokenAfterSuccessfulUse	PASSOU	Reutilização lança IllegalArgumentException
shouldDeleteUserDataUponOwnerRequest	PASSOU	findByEmail retorna Optional.empty()
shouldLinkCaregiverToPatientUsingSixCharacterInviteCode	PASSOU	linkedPatientEmail correto; token limpo
shouldRejectReusedCaregiverInviteToken	PASSOU	Erro: 'Codigo de vinculacao invalido'
shouldAllowPatientToRegisterOwnMedication	PASSOU	Medicamento registrado; email == paciente
shouldAllowLinkedCaregiverToRegisterMedicationForPatient	PASSOU	Medicamento visível no paciente vinculado
shouldRejectMedicationRegistrationForUnauthorizedUser	PASSOU	Exceção lançada para admin
shouldShowAllLogsOnlyForAdministrator	PASSOU	adminView.size() > irisView.size()
shouldAllowAdministratorToBlockAndUnblockUser	PASSOU	Bloqueio e desbloqueio funcionais

9.3 Análise dos Resultados

- Todos os controles de autenticação e autorização testados funcionam conforme especificado
- A lógica de bloqueio por tentativas e tokens de uso único está corretamente implementada
- A segregação de acesso entre roles (PACIENTE, CUIDADOR, ADMIN) é validada em cada operação
- O encadeamento por hash na auditoria foi verificado indiretamente pelos testes de criação de registros
- Os testes confirmam que dados de usuários excluídos não são mais recuperáveis via as interfaces de busca

REFERÊNCIAS

ABNT. Associação Brasileira de Normas Técnicas. **ABNT NBR ISO/IEC 27001:2022 – Segurança da informação, cibersegurança e proteção da privacidade – Sistemas de gestão da segurança da informação – Requisitos.** Rio de Janeiro: ABNT, 2022.

ABNT. Associação Brasileira de Normas Técnicas. **ABNT NBR ISO/IEC 27002:2022 – Segurança da informação, cibersegurança e proteção da privacidade – Controles de segurança da informação.** Rio de Janeiro: ABNT, 2022.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. **Lei Geral de Proteção de Dados Pessoais (LGPD).** Diário Oficial da União, Brasília, DF, 15 ago. 2018.

OWASP Foundation. *OWASP Top 10: The Ten Most Critical Web Application Security Risks.* Disponível em: <https://owasp.org/www-project-top-ten/>. Acesso em: 20 maio 2026.

Spring Documentation. *Spring Boot Reference Documentation.* Disponível em: <https://docs.spring.io/spring-boot/documentation.html>. Acesso em: 20 maio 2026.

Oracle Java Documentation. *Java Platform Documentation.* Disponível em: <https://docs.oracle.com/>. Acesso em: 20 maio 2026.

NIST Computer Security Resource Center. NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. *Digital Identity Guidelines – Authentication and Lifecycle Management (SP 800-63B).* Disponível em: <https://pages.nist.gov/800-63-3/sp800-63b.html>. Acesso em: 20 maio 2026.

SILVA, Luciano da. *Segurança da Informação para Sistemas Web.* São Paulo: Novatec Editora, 2021.

STALLINGS, William. *Criptografia e Segurança de Redes: princípios e práticas.* 7. ed. São Paulo: Pearson, 2018.

PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: uma abordagem profissional*. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson, 201